Token

Accesso sicuro ai servizi

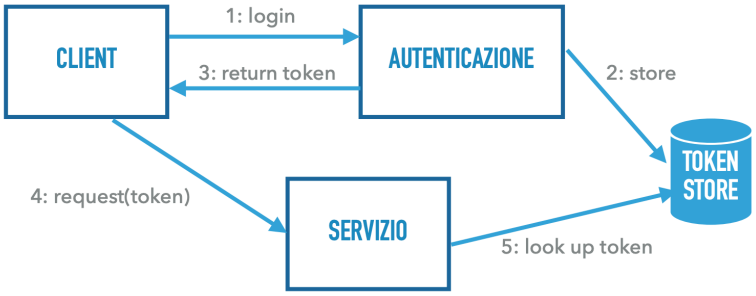
In un sistema distribuito, ogni richiesta inviata ad un servizio (lato server) deve essere controllata. Solo gli utenti autenticati e autorizzati devono poter accedere alle risorse. Tuttavia, sorgono due necessità contrastanti:

1. ogni richiesta deve essere verificata.
2. bisogna evitare di chiedere continuamente le credenziali (username e password) all'utente per ogni singola operazione. La soluzione è gestire la prova di identità in un unico punto (server) e rilasciare un "pass" temporaneo, cioè il **token.**

Flusso di autenticazione (workflow)

Il flusso tipico di un'autenticazione basata su token è il seguente:

1. **Login**: il client invia le credenziali al server di autenticazione.
2. **Emissione**: se le credenziali sono valide, il server genera un token e lo restituisce al client (eventualmente salvandone una copia o un riferimento in un "token store").
3. **Utilizzo**: per le richieste successive verso un servizio (ad esempio, leggere dati), il client allega il token.
4. **Verifica**: il servizio riceve il token e ne verifica la validità (consultando il token store o verificando la firma crittografica) prima di erogare il servizio. In questa fase potrebbe anche leggere i ruoli o permessi contenuti nel token.



Cos'è un token?

Un token è una credenziale di accesso che prova che il possessore è stato autorizzato. Si tratta di una sorta di "pass digitale temporaneo" che dimostra l'avvenuta autenticazione, che per un certo periodo di tempo si può accedere a certe risorse e che risolve quindi il problema di non chiedere ad ogni richiesta username e password.

Un token può essere:

- **Opaque** (opaco, incomprensibile): una stringa casuale (es. kPoPMRYrCEoYO6s5), che funge solo da identificativo per recuperare i dati reali dal server. In questo caso il server deve consultare un database (token store) e recuperare i dati associati.
- **Oggetto strutturato** (es. JWT): un oggetto che contiene al suo interno le informazioni (JSON Web Token).

Le informazioni tipicamente contenute in un token includono:

- Utente identificato.
- Ente che ha emesso il token (Issuer).
- Timestamp di emissione e durata (scadenza).
- Payload (dati aggiuntivi).
- Firma digitale per garantirne l'integrità.

Diagramma UML di sequenza

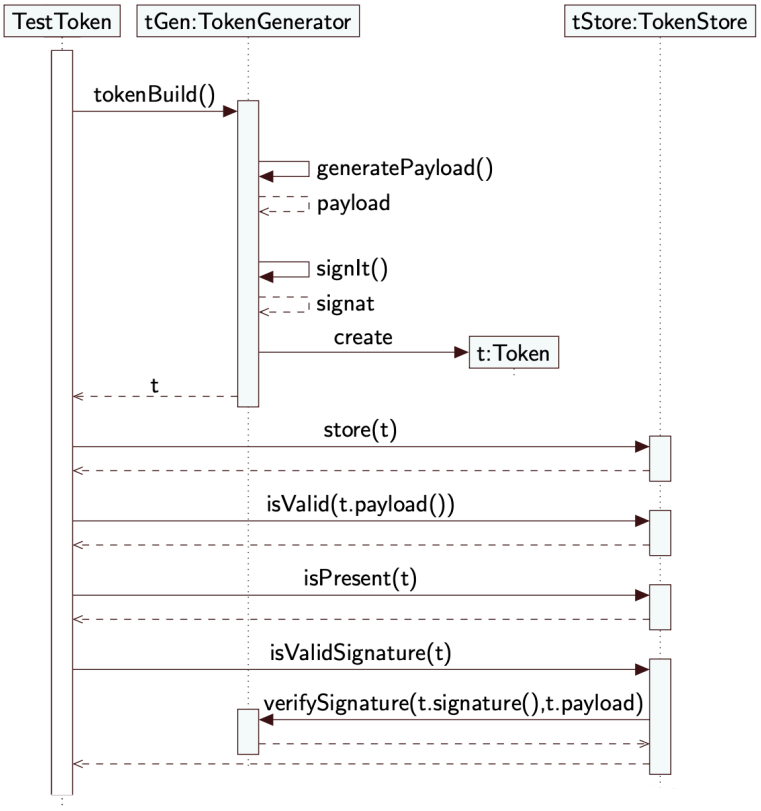
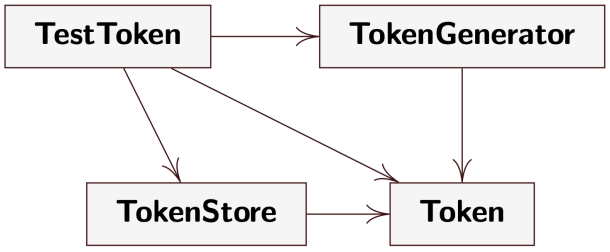


Diagramma UML delle classi



JSON Web Token (JWT)

Il formato JWT, pronunciato "jot", è uno standard definito dall'RFC 7519 per token di sicurezza auto-contenuti.

- **Auto-contenuto**: il token contiene al suo interno i “diritti” (claim) e gli attributi della sessione dell’utente. Il server crea l’oggetto JSON, lo invia al client e, quando lo riceve indietro, può recuperare lo stato della sessione leggendo direttamente il token, senza dover necessariamente accedere a un database centrale per ogni richiesta.
- **Sicurezza**: i JWT sono protetti crittograficamente (firmati) e possono essere criptati.

Claim standard (attributi)

Un JWT contiene dei campi standard definiti dal protocollo:

- **iss** (Issuer): chi ha creato il token.
- **sub** (Subject): l’identificativo univoco dell’utente.
- **aud** (Audience): i destinatari previsti del token.
- **nbf** (Not Before): il momento prima del quale il token non è valido.
- **iat** (Issued At): quando il token è stato creato.
- **exp** (Expiration Time): quando il token scade.
- **jti** (JWT ID): identificativo univoco del token.

Per impedire che un attaccante modifichi questi dati (ad esempio, cambiandosi l’ID utente), il token include una firma calcolata con una chiave segreta. Dato che il payload del token è leggibile (base64), la sicurezza non è nell’offuscamento, ma sta nella firma digitale. Se qualcuno modifica il campo Subject la firma non combacerà più e il server, verificando la firma, si accorge subito della manomissione.

Implementazione in Java (libreria Auth0)

Una libreria molto usata per il supporto alla generazione e verifica dei token è Auth0.

Al file di progetto pom.xml bisogna aggiungere la seguente dipendenza:

```
<dependency>
    <groupId>com.auth0</groupId>
    <artifactId>java-jwt</artifactId>
    <version>4.5.0</version>
</dependency>
```

Creazione del token

Per creare un token, si utilizza il pattern Builder. Si definiscono i claim e si firma il tutto con un algoritmo (es. RSA256 o HMAC).

```
String token = JWT.create()
                    .withIssuer("auth0-my-demo")
                    .withSubject("my-user-123")
                    .withIssuedAt(new Date())
                    .withExpiresAt(new Date(System.currentTimeMillis() + 3
600_000))
                    .withJWTId("token-xyz-123")
                    .sign(algorithm);
```

Il token creato contiene l'id di chi lo ha prodotto, l'id dell'utente, la data di creazione, la data di scadenza, l'id del token, e quindi la firma crittografica.

Il metodo `sign()` chiude il token con la firma crittografica, prendendo in input un'istanza che rappresenta l'algoritmo di crittografia per la firma e per la verifica:

```
Algorithm algorithm = Algorithm.RSA256(publicKey, privateKey);
```

Verifica del token

Per verificare un token ricevuto, si crea un JWTVerifier impostando le aspettative (es. l'issuer deve essere quello corretto). Se il token è stato manomesso o è scaduto, il metodo `verify()` lancerà un'eccezione:

```
JWTVerifier verifier = JWT.require(algorithm)
        .withIssuer("auth0-my-demo")
        .build();

// Lancia eccezione se il token non è valido
DecodedJWT decoded = verifier.verify(token);
System.out.println("Subject: " + decoded.getSubject());
```

Una volta verificato il token, è quindi possibile leggere i dati al suo interno, come ad esempio il Subject.